AI 时代的高效自学实践

通过在 Linux 虚拟机中安装 OpenCode、配置 DeepSeek API 和两个学习 skill，探索 TCP 连接建立、连接关闭、序号确认、半关闭与 TIME_WAIT，并比较文字讲解与 HTML 图解的学习效果。

项目	内容
姓名	刘顺文
学号	20262819
课程	Linux 原理与实践
实践日期	2026-11-08
实验环境	Windows 宿主机、VirtualBox 虚拟机、Ubuntu 26.04.1 LTS、x86_64
工具	OpenCode v2.2.6；DeepSeek API，模型 deepseek/deepseek-flash
运行环境	Node.js v26.13.0；npm 11.20.0
学习工具	help-me-understand；answer-me-with-html（配套 CLI v0.5.10）
截图材料	4 张安装与运行验证截图、20 张探索截图，共 24 张

01 | 实践目标与完成情况

本次实践按照“安装工具—配置模型—安装 skill—提出问题—查看图解—核对输出”的顺序进行。探索内容集中在 TCP 这一主题下的多个知识点，没有将同一主题的截图数量当作独立主题数量。

作业要求	本次完成情况
在 Linux 安装 OpenCode	已安装，命令路径与版本已验证
安装两个附件 skill	已安装到当前项目，OpenCode 返回的逐文件比对结果与附件一致
使用两个 skill 学习内容	已生成文字讲解、交互时序页面和 HTML 学习页面
不少于 10 张探索截图	正文包含 20 张真实探索截图，不含安装截图

02 | 环境准备与工具安装

2.1 安装 OpenCode

首次访问安装地址时出现“连接被对方重置”。宿主机浏览器可以通过 v2rayN 访问 GitHub，但虚拟机终端没有代理环境变量。开启宿主机的局域网代理入口后，通过 VirtualBox NAT 地址测试代理连通性，GitHub 返回 HTTP/2 200。

本次终端使用的代理设置如下，仅对当前终端及其子进程生效：

```
export http_proxy=http://10.0.2.2:10808
export https_proxy=http://10.0.2.2:10808
```

从官方仓库下载 OpenCode 安装脚本后，在副本中将安装目录改为 ~/software/opencode/bin，再执行安装。安装结束后加载 ~/.bashrc，验证实际命令位置和版本。

```
source ~/.bashrc
command -v opencode
opencode --version
```

实际输出路径为 /home/sevenlsw/software/opencode/bin/opencode，版本为 v2.2.6。

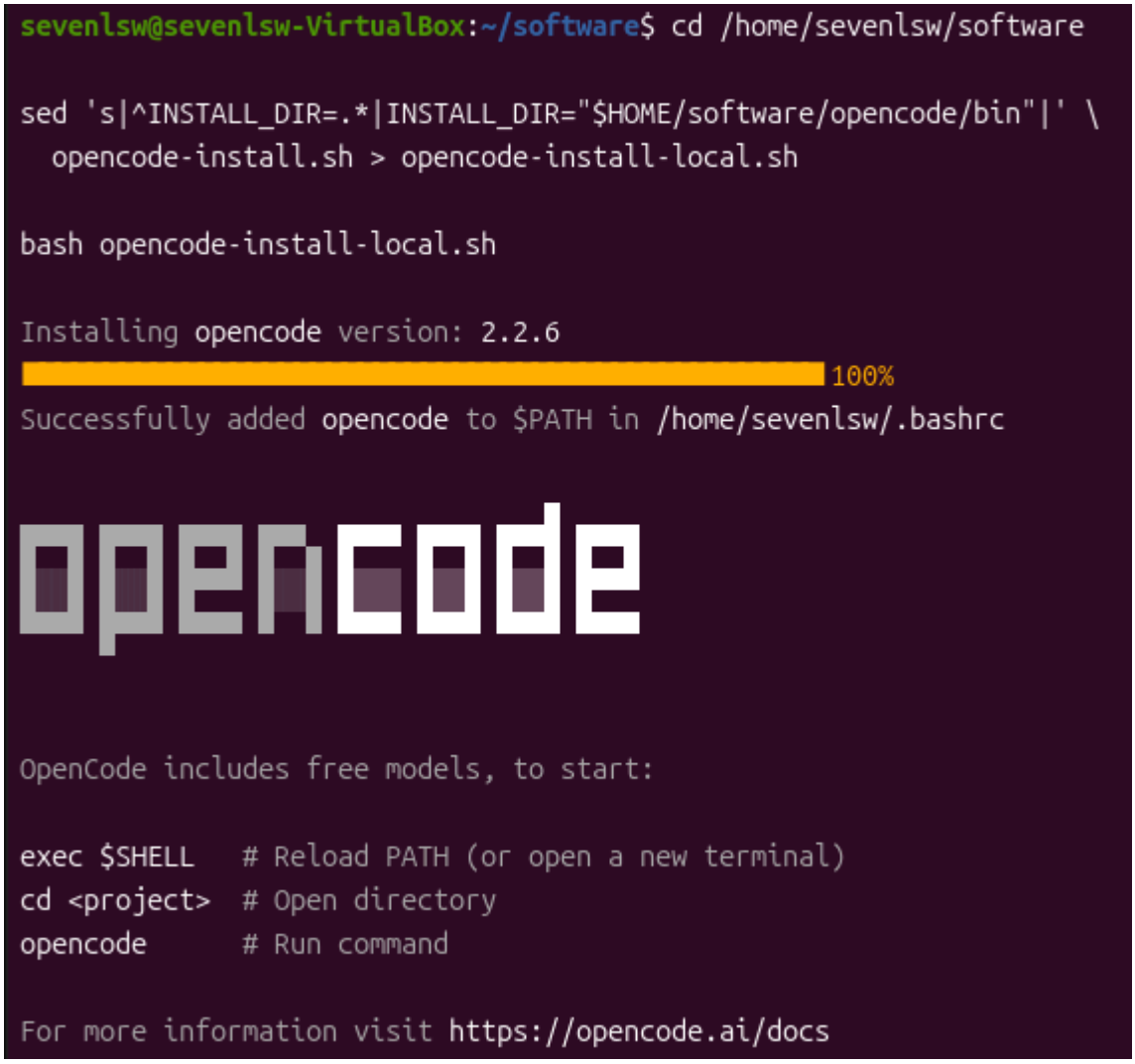


图1：OpenCode 下载和安装完成。

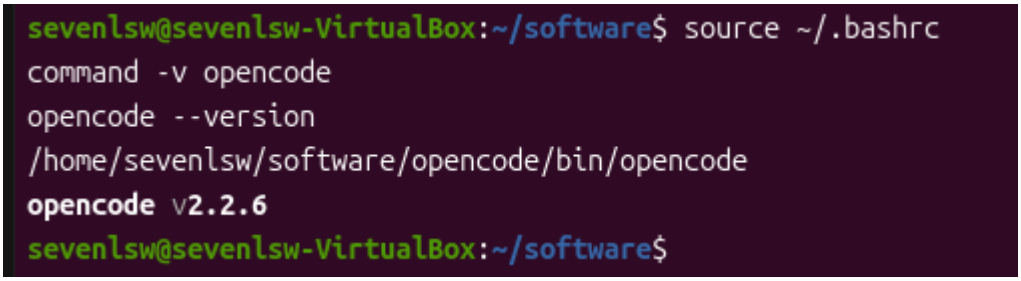


图2：核对 OpenCode 命令路径与版本。

2.2 接入 DeepSeek API

通过 opencode auth login 选择 DeepSeek 并在本机输入密钥。随后查看模型列表，明确指定 deepseek/deepseek-flash 发起简短请求，验证模型能够返回结果。报告不记录 API Key。

```
opencode models
opencode run -m deepseek/deepseek-flash "请只回复：DeepSeek API 连接成功"
```

实际运行显示 build · deepseek-flash，并返回“DeepSeek API 连接成功”。这一步验证了实际调用，而不只是配置项或模型名称可见。

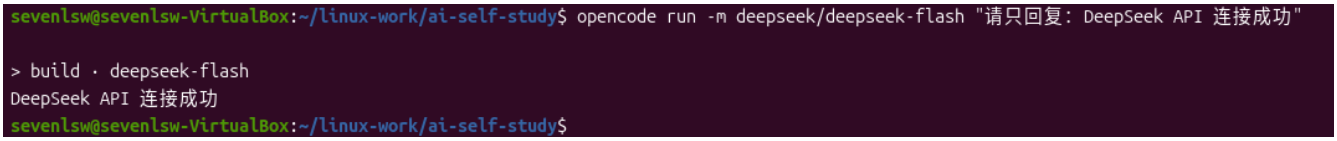


图3：指定 DeepSeek 模型并成功获得回复。

2.3 安装两个附件 skill

本次作业目录为 ~/linux-work/ai-self-study，软件安装目录与作业目录分开管理。两个 ZIP 解压到 attachments/ 后，找到的入口分别是：

```
attachments/help-me-understand/SKILL.md
attachments/answer-me-with-html/skills/answer-me-with-html/SKILL.md
```

随后使用以下形式的命令，让 OpenCode 阅读本地附件并核对安装：

```
opencode run -m deepseek/deepseek-flash "帮我安装这两个 skill help-me-understand answer-me-with-html。使用本地附件，安装到当前项目的 .opencode/skills/ 下，保留完整配套文件并核对结果。"
```

实际提示词还指定了两个源目录，并要求保留附件、避免覆盖已有文件。OpenCode 返回结果显示，两个 skill 此前已安装，本轮没有再次复制；diff -rq 比对无差异。最终入口为：

```
.opencode/skills/help-me-understand/SKILL.md
.opencode/skills/answer-me-with-html/SKILL.md
```

第一次依赖检查发现缺少 Node.js。安装 nvm 时 Git 克隆也出现连接重置，改用 METHOD=script 下载方式完成安装，然后执行 nvm install --lts。实际得到 Node.js v26.13.0 和 npm 11.20.0。最后验证 HTML 工具：

```
node .opencode/skills/answer-me-with-html/scripts/am.mjs --version
```

实际返回 0.5.10，说明配套 CLI 已可启动。

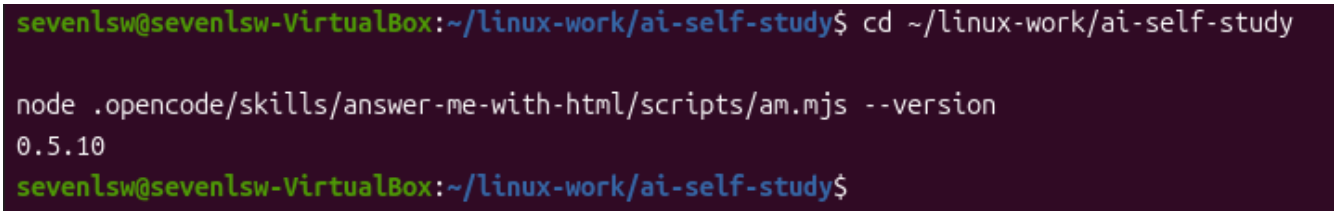


图4：answer-me-with-html 配套 CLI 版本验证。

03 | 使用 help-me-understand 探索 TCP

3.1 提问与文字讲解

本轮明确要求加载 help-me-understand，面向初学者解释 TCP 三次握手和四次挥手，通过生活类比与报文时序说明 SYN、ACK、FIN，解释握手次数、挥手次数和 TIME_WAIT，并给出三个不立即公布答案的自测题。

```
opencode run -m deepseek/deepseek-flash "使用 help-me-understand skill 帮我看懂 TCP 三次握手和四次挥手。通过生活类比和具体报文时序说明 SYN、ACK、FIN、半关闭和 TIME_WAIT，最后给出三个自测问题。将学习材料保存到 explorations/01-tcp-understand/。"
```

实际生成了 README.md 和 tcp-timeline.html。下面记录文字讲解的七张截图。

````
已按 `help-me-understand` 的思路整理好，完整材料在：

- `explorations/01-tcp-understand/README.md`：文字版，含报文时序、类比、常见误区和自测题
- `explorations/01-tcp-understand/tcp-timeline.html`：图解版，可点击“上一步 / 下一步”逐步查看握手和挥手

下面先讲核心内容。

---

## 一句话理解

**三次握手**：双方各自报出“我从哪个序号开始”，并确认对方收到了。
**四次挥手**：两个传输方向分别说“我发完了”，并各自确认。
**TIME_WAIT**：主动关闭方最后多等 2×MSL，保证最后一个 ACK 能送到，也让旧连接的迟到报文过期。

---

## 先分清 SYN、ACK、FIN

| 名称 | 作用 | 是否占序号 |
| --- | --- | --- |
| **SYN** | 同步序号，报出自己的初始序号 ISN | 占 1 个序号 |
| **ACK 标志位** | 表示“确认号字段有效”，即“我收到了” | 纯 ACK 不占序号 |
| **确认号（ack number）** | 下一个期待收到的序号 | 不占序号 |
| **FIN** | “我没有更多数据要发了”，只关闭发送方向 | 占 1 个序号 |

容易混的是：日常说的 “ACK” 有时指 **ACK 标志位**，有时指 **确认号**。例如 `ack=101` 的意思是“101 之前的字节我都收到了，下一个期待 101”。

---

## 三次握手：建立连接

用 RFC 9293 的教学序号：

```text
客户端 A                                  服务端 B
CLOSED                                    LISTEN
````

图5：探索截图 01：整体解释与 SYN、ACK、FIN 的含义。
````

```text
客户端 A                                    服务端 B
CLOSED                                      LISTEN

  |  SYN, seq=100  ---------------------------->  |
  |  A: SYN-SENT                          B: SYN-RECEIVED

  |  <----------------- SYN, ACK, seq=300, ack=101  |
  |  A: ESTABLISHED                       B: SYN-RECEIVED

  |  ACK, seq=101, ack=301  ---------------------->  |
  |  A: ESTABLISHED                       B: ESTABLISHED
```

每一步在做什么

1. **A →B: `SYN, seq=100`**
   A 说：“我从序号 100 开始。”SYN 占掉 100，所以 A 接下来的数据从 101 开始。A 进入 `SYN-SENT`。

2. **B →A: `SYN, ACK, seq=300, ack=101`**
   B 说：“我收到你的 100 了，下一个期待 101；同时我从 300 开始。”
   这个报文同时完成“确认 A 的 SYN”和“报出 B 自己的 SYN”，所以叫 `SYN+ACK`。

3. **A →B: `ACK, seq=101, ack=301`**
   A 说：“我收到你的 300 了，下一个期待 301。”
   纯 ACK 不占序号，所以 `seq` 仍是 101。A 发出后进入 `ESTABLISHED`，B 收到后也进入 `ESTABLISHED`。

生活类比：对讲机确认

- A：“喂，你能听到我吗？我从第 100 句开始说。”—SYN
- B：“能听到。我从第 300 句开始说，你从第 101 句开始。”—SYN+ACK
- A：“好，我知道你从第 301 句开始。”—ACK

这个类比只说明“双方各自报起点，并互相确认”。TCP 数的是**字节**，不是句子；现实对话也没有重传和乱序重组。

图6：探索截图 02：三次握手的报文与序号变化。

为什么通常需要三次？

关键在于：**两次不够让 B 确认 A 收到了 B 的 SYN。**

A 发出 SYN、B 回复 SYN+ACK 之后：

- A 已经知道 B 收到了自己的 SYN；
- A 也知道 B 从 300 开始；
- 但 **B 还不知道 A 有没有收到自己的 SYN**；
- B 也还不知道 A 现在是不是真的在参与连接，而不是一个迟到的旧 SYN。

A 最后的 ACK 正好补上这两个信息。三次之后，双方都完成了：

> 我发出自己的初始序号，并收到对方确认；
> 我收到对方的初始序号，并发出确认。

为什么不是四次？因为 B 的“确认 A 的 SYN”和“发送自己的 SYN”可以合并在一个 `SYN+ACK` 里。拆成两个报文也能工作，但多花一个报文，不会增加额外的同步保证。

四次挥手：关闭连接

```text
客户端 A                                    服务端 B
ESTABLISHED                                 ESTABLISHED

  |  FIN, ACK, seq=100, ack=300  ---------------->  |
  |  A: FIN-WAIT-1                        B: CLOSE-WAIT

  |  <---------------------- ACK, seq=300, ack=101  |
  |  A: FIN-WAIT-2                        B: CLOSE-WAIT
  |                                       B 仍可发送数据

  |  <------------------ FIN, ACK, seq=300, ack=101  |
  |  A: TIME-WAIT                          B: LAST-ACK
```

图7：探索截图 03：第三次确认的必要性与关闭流程。

```
客户端 A                                    服务端 B
ESTABLISHED                                 ESTABLISHED

  |  FIN, ACK, seq=100, ack=300  ---------------->  |
  |  A: FIN-WAIT-1                       B: CLOSE-WAIT

  |  <---------------------- ACK, seq=300, ack=101  |
  |  A: FIN-WAIT-2                       B: CLOSE-WAIT
  |                                      B 仍可发送数据

  |  <------------------ FIN, ACK, seq=300, ack=101  |
  |  A: TIME-WAIT                         B: LAST-ACK

  |  ACK, seq=101, ack=301  --------------------->  |
  |  A: TIME-WAIT                        B: CLOSED

  |  A 等待 2×MSL，然后进入 CLOSED
```

每一步在做什么

1. **A →B: `FIN, ACK, seq=100, ack=300`**
   A 说：“我没有更多数据要发了。”FIN 占 1 个序号，所以 B 要确认 101。A 进入 `FIN-WAIT-1`。
   注意：A 只是不能发**新数据**，仍然可以接收 B 的数据。

2. **B →A: `ACK, seq=300, ack=101`**
   B 说：“收到你的 FIN 了。”B 进入 `CLOSE-WAIT`，A 进入 `FIN-WAIT-2`。
   这时 B 的应用可能还有数据要发，所以 B 可以继续向 A 发送；A 仍然可以接收。

3. **B →A: `FIN, ACK, seq=300, ack=101`**
   B 说：“我也发完了。”B 进入 `LAST-ACK`，A 进入 `TIME-WAIT`。
   示例假设 B 中间没有发数据，所以 `seq` 仍是 300；如果 B 发了数据，序号会相应增加。

4. **A →B: `ACK, seq=101, ack=301`**
   A 说：“收到你的 FIN 了。”B 进入 `CLOSED`。A 还要在 `TIME-WAIT` 等 2×MSL。

图8：探索截图 04：四次挥手中双方的状态变化。

```
### 生活类比：两个人结束谈话

- A：“我没话说了。”—FIN
- B：“好。”—ACK
- B 可能还有话要说，于是继续说；A 仍然听着。
- B：“我也说完了。”—FIN
- A：“好。”—ACK
- A 在门口多等一会儿再走，以防 B 没听到最后那声“好”。—TIME_WAIT

### 为什么通常需要四次？

因为 TCP 是**全双工**的：A→B 和 B→A 是两个独立方向。FIN 只关闭发送方向，不关闭接收方向。

- A 发 FIN，只关闭 A→B；
- B 先回 ACK，确认收到 A 的 FIN；
- 但 B 可能还有数据没发完，不能立刻发自己的 FIN；
- 等 B 发完，B 再发 FIN；
- A 再 ACK 这个 FIN。

所以逻辑上是四个动作：**FIN、ACK、FIN、ACK**。

但“四次挥手”不一定等于抓包里一定有四个包。如果 B 没有数据要发、应用也立刻关闭，B 可以把 ACK 和 FIN 合并成一个 `FIN, ACK`，看起来就是三个包。关键仍是：**两个方向分别关闭，每个方向都有 FIN 和 ACK。**

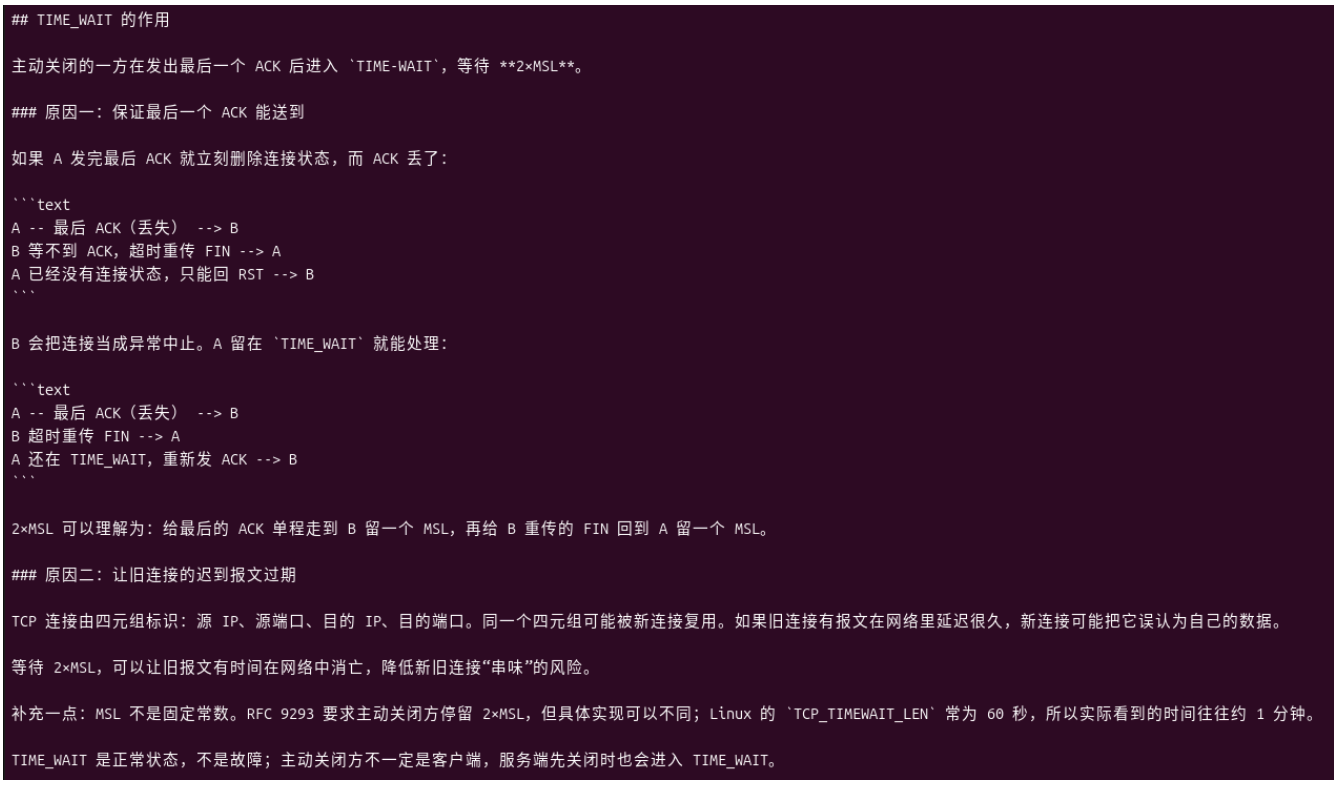
---

## TIME_WAIT 的作用

主动关闭的一方在发出最后一个 ACK 后进入 `TIME-WAIT`，等待 **2×MSL**。

### 原因一：保证最后一个 ACK 能送到
```

图9：探索截图 05：半关闭及 ACK 与 FIN 可能合并。



```
## TIME_WAIT 的作用

主动关闭的一方在发出最后一个 ACK 后进入 `TIME-WAIT`，等待 **2×MSL**。

### 原因一：保证最后一个 ACK 能送到

如果 A 发完最后 ACK 就立刻删除连接状态，而 ACK 丢了：

```text
A -- 最后 ACK（丢失） --> B
B 等不到 ACK，超时重传 FIN --> A
A 已经没有连接状态，只能回 RST --> B
```

B 会把连接当成异常中止。A 留在 `TIME_WAIT` 就能处理：

```text
A -- 最后 ACK（丢失） --> B
B 超时重传 FIN --> A
A 还在 TIME_WAIT，重新发 ACK --> B
```

2×MSL 可以理解为：给最后的 ACK 单程走到 B 留一个 MSL，再给 B 重传的 FIN 回到 A 留一个 MSL。

原因二：让旧连接的迟到报文过期

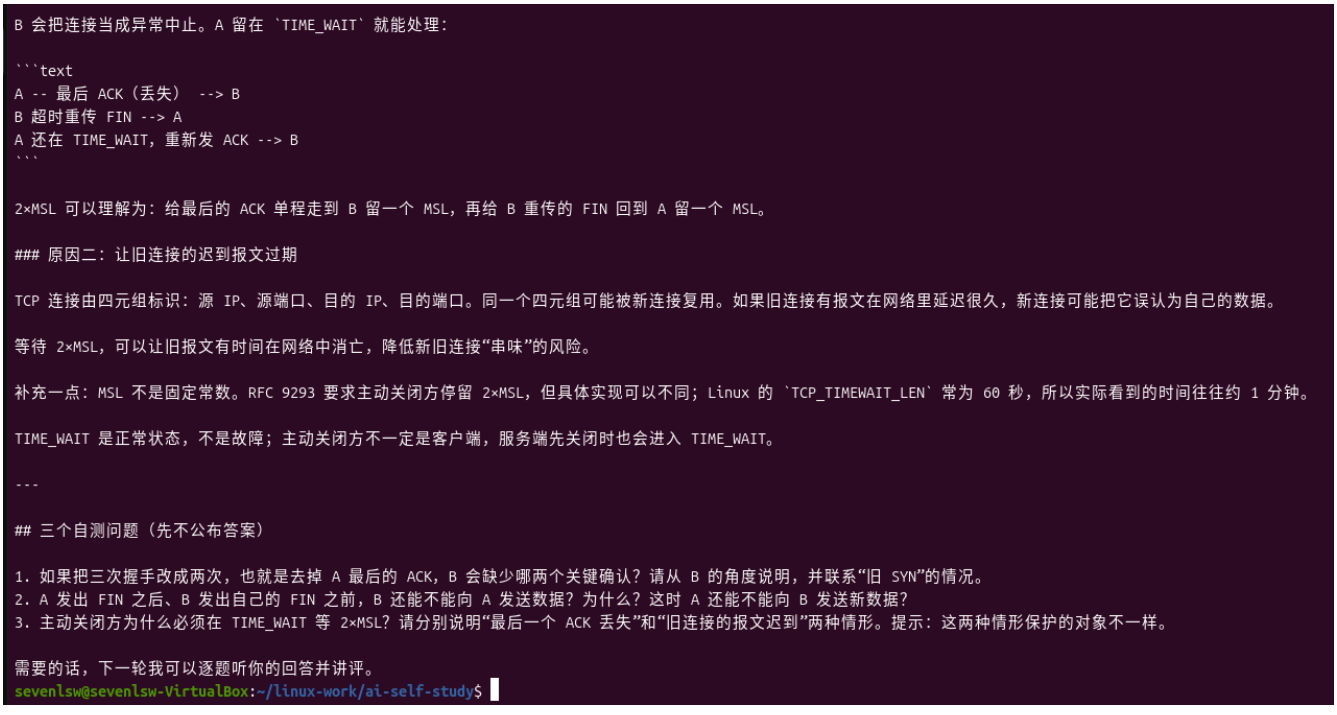
TCP 连接由四元组标识：源 IP、源端口、目的 IP、目的端口。同一个四元组可能被新连接复用。如果旧连接有报文在网络里延迟很久，新连接可能把它误认为自己的数据。

等待 2×MSL，可以让旧报文有时间在网络中消亡，降低新旧连接“串味”的风险。

补充一点：MSL 不是固定常数。RFC 9293 要求主动关闭方停留 2×MSL，但具体实现可以不同；Linux 的 `TCP_TIMEWAIT_LEN` 常为 60 秒，所以实际看到的时间往往约 1 分钟。

TIME_WAIT 是正常状态，不是故障；主动关闭方不一定是客户端，服务端先关闭时也会进入 TIME_WAIT。
```

图10：探索截图 06：TIME_WAIT 与最后 ACK 丢失后的处理。



```
B 会把连接当成异常中止。A 留在 `TIME_WAIT` 就能处理：

```text
A -- 最后 ACK（丢失） --> B
B 超时重传 FIN --> A
A 还在 TIME_WAIT，重新发 ACK --> B
```

2×MSL 可以理解为：给最后的 ACK 单程走到 B 留一个 MSL，再给 B 重传的 FIN 回到 A 留一个 MSL。

原因二：让旧连接的迟到报文过期

TCP 连接由四元组标识：源 IP、源端口、目的 IP、目的端口。同一个四元组可能被新连接复用。如果旧连接有报文在网络里延迟很久，新连接可能把它误认为自己的数据。

等待 2×MSL，可以让旧报文有时间在网络中消亡，降低新旧连接“串味”的风险。

补充一点：MSL 不是固定常数。RFC 9293 要求主动关闭方停留 2×MSL，但具体实现可以不同；Linux 的 `TCP_TIMEWAIT_LEN` 常为 60 秒，所以实际看到的时间往往约 1 分钟。

TIME_WAIT 是正常状态，不是故障；主动关闭方不一定是客户端，服务端先关闭时也会进入 TIME_WAIT。

三个自测问题（先不公布答案）

1. 如果把三次握手改成两次，也就是去掉 A 最后的 ACK，B 会缺少哪两个关键确认？请从 B 的角度说明，并联系“旧 SYN”的情况。
2. A 发出 FIN 之后、B 发出自己的 FIN 之前，B 还能不能向 A 发送数据？为什么？这时 A 还能不能向 B 发送新数据？
3. 主动关闭方为什么必须在 TIME_WAIT 等 2×MSL？请分别说明“最后一个 ACK 丢失”和“旧连接的报文迟到”两种情形。提示：这两种情形保护的对象不一样。

需要的话，下一轮我可以逐题听你的回答并讲评。
sevenlsw@sevenlsw-VirtualBox:~/linux-work/ai-self-study$
```

图11：探索截图 07：迟到报文解释与三个自测问题。

3.2 打开交互时序图

```
xdg-open explorations/01-tcp-understand/tcp-timeline.html
```

浏览器中可以点击“上一步 / 下一步”，逐步显示报文、连接状态和当前步骤说明。握手部分展示三个报文及完成总结，挥手部分展示四个报文及 TIME_WAIT 等待阶段。



图12：探索截图 08：交互页面首页与基本概念。



图13：探索截图 09：客户端发送 SYN。



图14：探索截图 10：服务端回复 SYN 与 ACK。



图15：探索截图 11：客户端发送第三次 ACK。



图16：探索截图 12：双方完成连接建立。



图17：探索截图 13：主动关闭方发送 FIN。



图18：探索截图 14：对方确认 FIN，连接处于半关闭阶段。



图19：探索截图 15：另一方向发送 FIN。



图20：探索截图 16：主动关闭方回复最后的 ACK。



图21：探索截图 17：TIME_WAIT 等待阶段。

3.3 对知识点的理解

三次握手用于同步并确认两个方向的初始序号。以客户端初始序号为 100、服务端为 300 的示例来说，三个报文依次为 SYN(seq=100)、SYN+ACK(seq=300, ack=101) 和 ACK(seq=101, ack=301)。SYN 消耗一个序号，纯 ACK 不消耗序号。第三个报文使服务端获得对自身 SYN 的确认。

TCP 是全双工连接。一个方向发送 FIN 表示该方向没有更多数据，不等于两个方向立即全部关闭。另一方确认 FIN 后，仍可能继续发送尚未完成的数据；等它也关闭发送方向时，再发送自己的 FIN。因此通常表现为四个挥手报文，但 ACK 和 FIN 可能合并，不能机械地认为抓包结果一定有四个包。

TIME_WAIT 保留关闭过程中的必要状态，使一方在收到对端重传的 FIN 时仍可再次回复 ACK；同时，通过等待降低旧连接迟到或重复报文对后续连接的影响。

04 | 使用 answer-me-with-html 生成学习页面

4.1 提问与生成结果

第二轮加载 answer-me-with-html，要求使用配套工具生成中文学习页面，包含报文时序、双方状态、序号消耗、半关闭、TIME_WAIT、挥手报文合并和自测题。

```
opencode run -m deepseek/deepseek-flash "使用 answer-me-with-html skill 帮我解释 TCP 三次握手和四次挥手。使用配套工具生成中文 HTML 学习页面，包含报文时序、状态变化、序号消耗、半关闭、TIME_WAIT 和自测题，保存到 explorations/02-tcp-html/。"
```

实际页面为 explorations/02-tcp-html/tcp-handshake-and-wave.html。打开后可看到工具标识、时序图、状态表，以及主题、明暗和字号等阅读设置。



图22：探索截图 18：第二个 skill 生成的页面与握手时序。

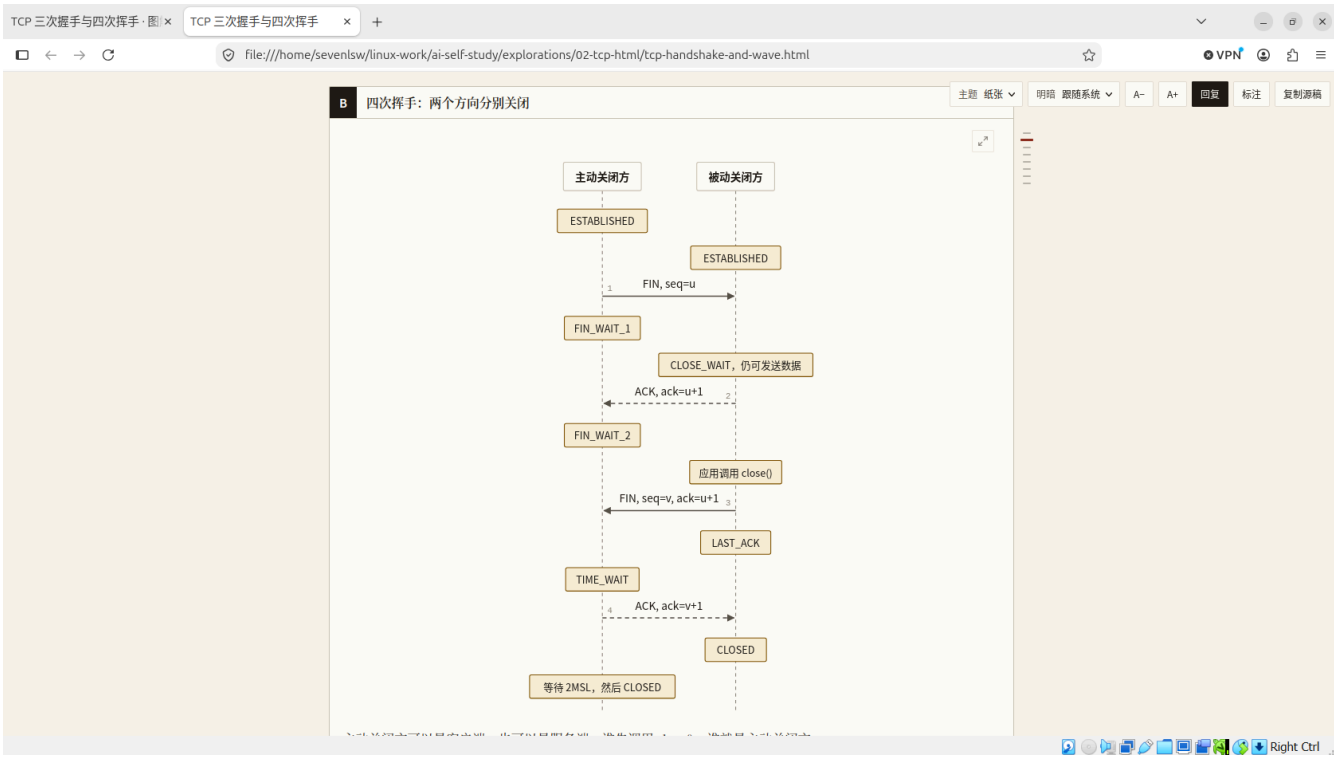


图23：探索截图 19：HTML 页面中的四次挥手时序。

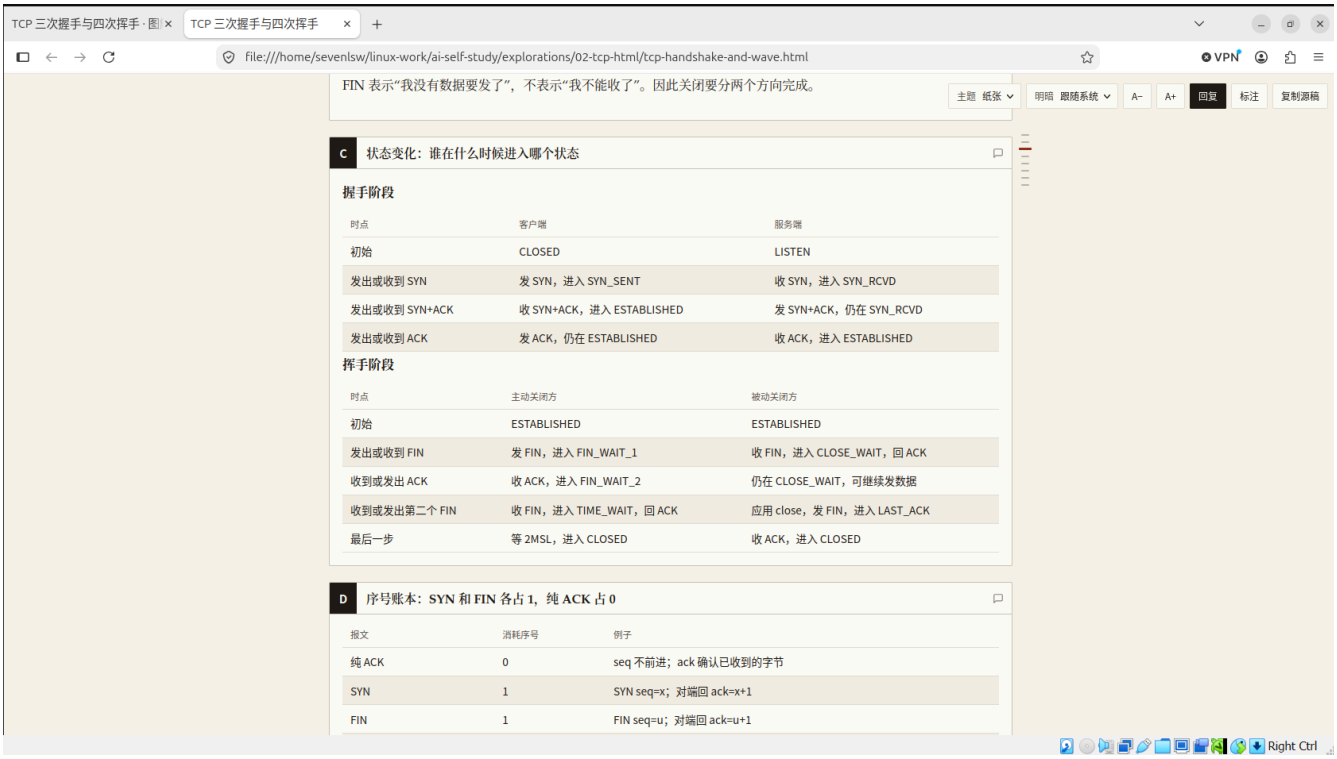


图24：探索截图 20：HTML 页面中的状态变化与序号消耗表。

4.2 两种输出的比较

比较项	help-me-understand 本轮输出	answer-me-with-html 本轮输出
组织方式	文字讲解、类比、自测及独立交互图解	由配套 CLI 渲染的分面板学习页面
观察方式	按步骤点击，观察当前报文和状态	对照完整时序图、状态表和说明
学习用途	拆解过程，逐步理解因果关系	将知识点汇总为便于浏览的页面

两种 skill 都能够输出图文材料，不能简单地将它们区分为“只能输出文字”和“只能输出网页”。本轮的差别主要在于讲解组织方式和页面呈现方式。

05 | 核对 AI 输出与问题处理

5.1 实际遇到的问题

问题	排查与处理	结果
下载地址连接被重置	核对代理环境变量，开启宿主机局域网代理并测试	GitHub 请求成功，继续下载完成
models 命令不接受提供商位置参数	改用 opencode models 查看模型列表	获取实际 DeepSeek 模型名称
HTML 工具缺少 Node.js	使用 nvm 安装 Node.js 并验证 CLI	CLI 返回 0.5.10
nvm 的 Git 克隆失败	改用 METHOD=script 方式安装	nvm 与 Node.js 安装成功
HTML 首轮末尾出现无法回答的 Web Search 请求	先打开已生成文件检查，再要求只根据现有知识修改	页面可打开；后续返回重新渲染成功

5.2 对不准确表述的纠正

AI 最初生成“TIME_WAIT 只属于主动关闭方”这一标题，表述过于绝对。常见的非同时关闭流程中，主动关闭方进入 TIME_WAIT；同时关闭时，双方都可能进入 TIME_WAIT。已让 OpenCode 修改源稿并重新生成页面，返回结果显示相应标题、正文和状态表前的说明已修改。

“保证最后一个 ACK 能送到”也应改为“保留状态，以便收到重传 FIN 时再次回复 ACK”。TCP 的重传机制不能保证任意网络故障下报文必定送达。

后续检查还发现两处需要在理解时注意：自测题应问“为什么不能发出最后一个 ACK 后立即删除连接状态”，而不是“收到最后一个 ACK”；常见 Linux 实现中 TIME_WAIT 默认约为 60 秒，这属于实现中的时间设置，不能据此声称测得网络报文的最大生存时间为 30 秒。以上两点在本报告中采用了纠正后的解释，未将尚无执行结果的最后一轮页面修正记为完成。

截图记录的是实践时实际看到的页面，因此可能保留修改前的文字。核对记录与解释共同说明了从生成、检查到修正的学习过程。

06 | 实践总结

本次实践完成了 OpenCode 安装、DeepSeek API 接入、两个本地 skill 的安装与依赖准备，并围绕 TCP 的连接建立、连接关闭、序号确认、半关闭和 TIME_WAIT 进行了学习。文字解释帮助梳理概念，交互时序图帮助观察每一步报文与状态变化，HTML 页面则便于集中对照和复习。

实践也说明，使用 AI 学习需要明确问题、运行工具并核对结果。模型生成的页面可以正常显示，并不意味着其中每句话都准确。对 TIME_WAIT 的适用场景、重传能力和发送接收方向进行检查，是本次自学中比单纯获取答案更有价值的一部分。